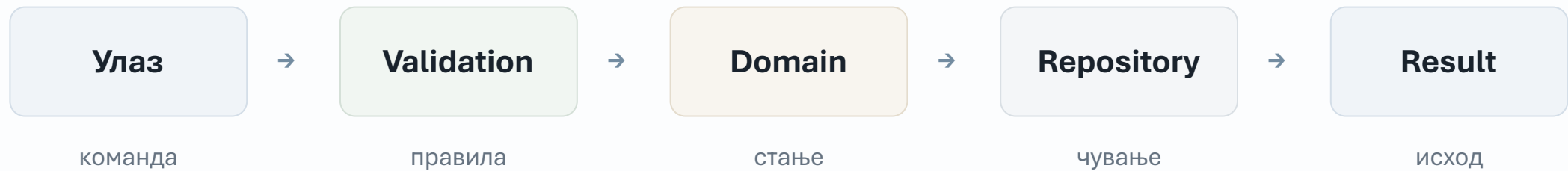


Пословна логика и слој случајева употребе

Распоређивање пословних правила и организација сценарија случаја употребе.

Случај употребе и пословни исход

Случај употребе оркестрира валидаторе, доменске сервисе и портове без зависности од корисничког интерфејса и базе.



НАПОМЕНА

Случај употребе има јасан улаз, експлицитан резултат и контролисане зависности.

Модел и пословна правила

Модел представља стање, док валидатори и доменски сервиси проверавају ограничења и спроводе пословна правила.

Reservation садржи податке о периоду и стању. **ReservationValidator** проверава дозвољене вредности, а доменски сервис спроводи правила сценарија.

НАПОМЕНА

Случај употребе координише валидаторе, доменске сервисе и портове.

Factory метод и валидно стање

Factory метод користи валидацију приликом креирања модела и враћа експлицитан Result.

```
1 public sealed class ReservationMaker
2 {
3
4     public Result<Reservation> Create(DateTime from, DateTime to)
5     {
6         if (from >= to)
7             return Result<Reservation>.Fail("InvalidPeriod");
8         return Result<Reservation>.Ok(new Reservation(from, to));
9     }
10 }
```

НАПОМЕНЕ

Конструктор може бити приватан.

Factory спречава очигледно невалидно стање.

Метод ентитета не треба да имају пословну логику – она остаје у сервисима.

Ограничења валидног модела

Валидатор проверава ограничења вредности пре покретања пословног сценарија.

01

Период

почетак мора бити пре краја

02

Стање

отказана резервација се не
отказује поново

03

Време

прошла резервација се не
мења као будућа

Очекивани пословни неуспех

Заузет термин представља исправно одбијен захтев са стабилним кодом исхода.

Систем ради исправно када стабилно врати `OverlappingReservation` и не промени стање које не сме да се промени.

НАПОМЕНА

Очекивани пословни неуспех мора имати експлицитан и тестабилан исход.

Result као модел исхода

Result експлицитно представља успех или очекивани пословни неуспех.

```
1 public sealed record Result(bool Success, string? Error)
2 {
3     public static Result Ok() => new(true, null);
4     public static Result Fail(string error) => new(false,
5         error);
6 }
7
8 public sealed record Result<T>(bool Success, T? Value,
9     string? Error);
```

НАПОМЕНЕ

Код неуспеха је стабилан.

Клијент не мора да тумачи изузетак.

Тест може директно да провери очекиван исход.

Избор модела исхода

Модел исхода зависи од врсте грешке и места на коме она настаје.

Ситуација	Препоручени модел
Неважећи пословни захтев	Result и стабилан код неуспеха
Ентитет није пронађен	NotFound исход или Result код
База недоступна	инфраструктурна грешка
Нарушена доменска претпоставка	експлицитан неуспех у пословном слоју

Сервис случаја употребе

Сервис координише сценарио преко доменских сервиса и уговора.

```
1 public sealed class ReservationService
2 {
3     private readonly IReservationRepository _repository;
4     private readonly IClock _clock;
5
6     public Result Reserve(ReservationRequest request)
7     {
8         if (request.From <= _clock.UtcNow)
9             return Result.Fail("ReservationMustBeInFuture");
10        if (_repository.HasOverlap(request.EquipmentId,
11            request.From, request.To))
12            return Result.Fail("OverlappingReservation");
13        return Result.Ok();
14    }
15 }
```

НАПОМЕНЕ

Use-case проверава ток операције.

Repository је уговор, не конкретна база.

Време је зависност која се контролише.

Dependency Injection

Променљиве зависности прослеђују се споља преко јасних уговора.

01

IClock

контролише време

02

Repository

складиште као уговор

03

Service

оркестрација use-case-a

04

Composition root

бира конкретне имплементације

Положај пословних правила

Контролер преводи улаз, позива случај употребе и мапира резултат.

CONTROLLER

прима захтев

валидација формата

позива application use-case

мапира Result у HTTP или UI одговор

USE-CASE

проверава пословни ток

позива доменске операције

користи портове

враћа експлицитан исход

Граница случаја употребе

Случај употребе представља операцију са јасним улазом, пословним исходом и зависностима.

Добар use-case може да се објасни од захтева до резултата: шта корисник покушава, која правила важе, које стање се мења и који исход систем враћа.

НАПОМЕНА

Случај употребе је јединица пројектовања и одбране функционалности.

Распоређивање правила по слојевима

Положај правила зависи од података које користи и стања које штити.

01

Entity

инваријанте сопственог стања

02

Domain service

правило више доменских објеката

03

Application service

оркестрација use-case-a

04

Infrastructure

технички адаптер без пословне одлуке

Кодови исхода као део уговора

Стабилни кодови омогућавају клијенту да поуздано обради очекивани неуспех.

Код	Значење
InvalidPeriod	почетак није пре краја
ReservationMustBeInFuture	термин није у будућности
OverlappingReservation	период се преклапа
AlreadyCancelled	операција је већ извршена

Контролна тачка P2

Функционално језгро мора бити разумљиво и образложиво.

2

кључна use-case-a

До контролне тачке треба имати најмање два use-case-a са експлицитним улазима, резултатима и јасним границама одговорности.

Захтеви контролне тачке P2

Провера обухвата централно понашање, границе одговорности и стабилне исходе.

- 01 Имплементирати најмање два кључна use-case-а.
- 02 Пословна правила не расипати по controller-у, менију и repository-ју.
- 03 За очекиване неуспехе дефинисати стабилне кодове или типове резултата.
- 04 Спољне зависности уводити кроз уговоре и composition root.
- 05 У PR-у описати најмање једну архитектонску одлуку.

Самостални задатак: замена bool/null вредности Result типом

Метода треба да врати стабилан код и податке потребне за обраду исхода.

```
1  // пре
2  bool Reserve(Request request);
3
4  // после
5  Result Reserve(Request request);
6
7  // тест проверава
8  Assert.That(result.Success, Is.False);
9  Assert.That(result.Error, Is.EqualTo("OverlappingReservatio
10      n"));
```

НАПОМЕНЕ

Рефакторисати једну методу са bool/null исходом.

Додати два мала теста или демонстрациона сценарија.

Образложити нови стабилан код неуспеха.

Границе случаја употребе

Јасан улаз, пословно правило и експлицитан резултат омогућавају проверу сценарија.

01

Input

команда или захтев

02

Policy

пословно понашање

03

Output

експлицитан исход

Резиме

Пословна логика припада доменским сервисима и случајевима употребе, уз одвојену валидацију модела.

Use-case није техничка основа, већ контролисан ток који повезује захтев, доменска правила, зависности и стабилан резултат.

НАПОМЕНА

Овим се поставља основа за аутоматизовано тестирање.

Припрема за наредну вежбу

Припремити сценарије који се могу проверити NUnit тестовима.

- 01 Изабрати два use-case-а у пројекту.
- 02 За сваки дефинисати улаз, пословне исходе и стабилне кодове грешке.
- 03 Проверити да ли доменски објекти штите своје инваријанте.
- 04 Изместити пословна правила из UI/API слоја.
- 05 Припремити сценарије за NUnit тестове.